

EduStreamiX — Class 12 Learning Platform

May 2026 · Full-Stack Web App · Node.js · Express · Google APIs · Razorpay

Anshumaan Sai Patnaik

Overview

EduStreamiX is a full-stack educational platform for Class 12 students. It organises study resources by educational board and subject, delivers them through a clean, distraction-free interface, and adds an AI-powered practice-test system that both generates exam-style questions and evaluates the student's written answers. Premium access is unlocked through an integrated payment flow, and the application is deployed and running live.

The Problem

Class 12 students preparing for board exams usually rely on scattered YouTube playlists and generic websites — environments full of distractions, with no structure around their specific board and subject, and no way to practise under exam-like conditions. Quality practice papers and timely feedback on written answers are especially hard to come by.

EduStreamiX addresses this by bringing curated, board- and subject-specific resources into one focused interface, and by using AI to generate board-pattern practice tests on demand and grade the student's answers with constructive feedback — replacing the need for manually prepared papers and an examiner.

Technology Stack

Node.js + Express	Application server, routing, and server-side rendering of the pages.
EJS	Templates for the home, study, and payment views.
Google Drive API	Used as a JSON content store for syllabus/resource data, accessed via a Google service account.
Google Generative AI (Gemini)	Generates practice tests and evaluates written answers.
Razorpay	Payment gateway for unlocking premium access.
JWT + cookie-parser	Signed, HTTP-only session cookie that gates premium content.
crypto · dotenv · cors	Payment-signature hashing, environment secrets, and cross-origin handling.

Architecture

The backend is a modular Express application. Payment and access-control logic lives in the app setup, while the learning features are split into **routes**, **controllers**, and a **services** layer that wraps the external Google integrations. Notably, the app has no traditional user database — access is granted entirely through a signed token, which keeps the system stateless and simple to deploy.

METHOD	PATH	PURPOSE
POST	/create-order	Create a Razorpay payment order.
POST	/verify-payment	Verify the payment signature and issue the access token.
GET	/	Home page (gated behind a valid token).
GET	/study	Board- and grade-specific resources from Google Drive.
POST	/study/generate-test	Generate an AI practice test (MCQ or theory).
POST	/study/evaluate-theory	AI-grade submitted theory answers.

Key Functionality

Premium access: payment verification & token gating

Unlocking the platform is a two-step flow. The client first requests an order from `/create-order`; after the user pays, Razorpay returns a payment ID and a signature. The server independently recomputes that signature with an HMAC-SHA256 hash of the order and payment IDs keyed by the secret, and only proceeds if it matches — this is what proves the payment is genuine and untampered. On success it issues a 30-day JWT in an HTTP-only cookie.

```
// Verify Razorpay's signature, then issue a 30-day premium cookie
const generated_signature = crypto
  .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
  .update(razorpay_order_id + "|" + razorpay_payment_id)
  .digest("hex");

if (generated_signature === razorpay_signature) {
  const token = jwt.sign({ premium: true }, process.env.SECRET_TOKEN, { expiresIn: "30d" });
  res.cookie("session_token", token, { httpOnly: true, sameSite: "lax", maxAge: 30*24*60*60*1000 });
  return res.json({ success: true, message: "Payment Verified & Access Granted" });
}
return res.status(400).json({ success: false, message: "Invalid Signature" });
```

A single piece of middleware sits in front of every page: if there is no cookie, or the token fails verification, the user is shown the payment page instead of the content. This turns the whole site into premium content behind one stateless gate.

Content delivery through Google Drive

Rather than running a separate database, the syllabus and resource data is stored as JSON files in a private Google Drive folder and read at request time through a Google service account. Each board's collection is fetched once and then held in an in-memory cache, so repeat requests are served instantly without another network round-trip to Drive.

```
// Treat a Drive folder of JSON files as a content store, cached in memory
async function getCollection(collectionName) {
  if (cache[collectionName]) return cache[collectionName];    // cache hit

  const file = await findFile(`${collectionName}.json`);
  if (!file) throw new Error(`File not found: ${collectionName}.json`);

  const response = await drive.files.get(
    { fileId: file.id, alt: 'media' },
    { responseType: 'text' }
  );

  cache[collectionName] = JSON.parse(response.data);           // store for next time
  return cache[collectionName];
}
```

The study route then loads the right collection based on the requested board (SSC, CBSE, or ISC), filters it to the chosen grade, and renders the subject list — falling back to an empty state when nothing matches.

AI-powered practice tests

The test generator builds a detailed prompt for the Gemini model and supports two modes. In **MCQ mode** it asks for multiple-choice questions with options, the correct index, and an explanation; in **theory mode** it produces board-pattern written questions with ideal answers, choosing the number of questions and the marks per question dynamically based on the question type and subject. Each test also asks the model to estimate how long a student should take, which drives an on-screen timer.

Because language models often wrap JSON in markdown fences, the response is run through a small cleanup step before parsing, and parsing is wrapped in error handling so a malformed AI reply returns a clean error rather than crashing the request. The same approach powers **answer evaluation**: submitted answers are batched into one prompt alongside the ideal answers and max marks, and the model returns awarded marks and short feedback per question — effectively an automated examiner.